

★ SPRING BOOT ACTUATORS

◆ 1. Understanding the Need for Spring Boot Actuators

When an application runs inside an IDE, developers have access to logs, profilers, debuggers, and all internal modules, so diagnosing an issue is relatively easy.

But once the same application is packaged as a **JAR** and deployed to a real **production server**, you completely lose visibility into what's happening inside.

You cannot directly see:

- How much memory the app is using
- Whether the database connection is working
- How many HTTP requests are coming
- Which endpoints are mapped
- Whether beans are loaded correctly
- Why performance is slowing down
- Whether a thread is stuck or deadlocked

Production becomes a **black box**, and any small issue can impact thousands of users.

To solve this problem, Spring Boot Actuators provide a **set of built-in endpoints** which reveal internal runtime information safely and consistently, without writing monitoring code manually.

Actuators act like **diagnostic windows** into a running Spring Boot application, giving DevOps, developers, and monitoring tools the visibility required to keep the system reliable.

◆ 2. What Really Happens When You Deploy a Spring Boot App (Expo File Concept)

When you build a Spring Boot project, Maven produces a single packaged file:

```
myapp-1.0.jar
```

Inside this JAR (your “expo file”), Spring Boot bundles:

- All your compiled classes
- All dependencies (inside BOOT-INF/lib)
- Embedded Tomcat server
- Configuration files
- Resources and templates

Once deployed, you cannot manually open or inspect anything inside that JAR.

You cannot see what beans got created, what configurations loaded, or which web endpoints are active.

Actuators allow you to **observe the internal state of this sealed running JAR** through well-defined endpoints like:

- `/actuator/health`
- `/actuator/metrics`
- `/actuator/env`
- `/actuator/mappings`

They let you “look inside” the expo file without modifying or redeploying it.

◆ 3. Enabling Actuators and Why So Many JARs Get Added

When you add:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Spring Boot automatically includes several supporting JARs such as:

- Micrometer (metrics engine)
- Health indicator libraries
- Auto-configuration modules
- JSON serializers

This happens because Spring Boot uses the **starter dependency pattern**, where one starter pulls multiple required libraries.

This is intentional and beneficial—Actuators rely on many internal components to collect metrics, run health checks, analyze threads, and expose endpoints safely.

So seeing many new JARs in your dependency tree is completely normal.

◆ 4. Exposing Actuator Endpoints and Why Only Two Are Visible Initially

After adding the starter, visit:

```
http://localhost:8080/actuator
```

You will see only:

- `health`
- `info`

Spring intentionally hides all other endpoints for security reasons.

To expose everything:

```
management.endpoints.web.exposure.include=*
```

To expose only selected ones:

```
management.endpoints.web.exposure.include=health,info,metrics,beans
```

This ensures that only trusted information is exposed depending on your environment (dev, prod, staging).

5. The `/health` Endpoint — The Primary Indicator of Application State

The `health` endpoint is the most frequently used actuator endpoint.

It tells whether your application is **UP** or **DOWN** based on internal checks like:

- Database availability
- Disk space
- External service checks
- Redis/Mongo/Kafka connections

Simple output:

```
{"status":"UP"}
```

If you enable detailed view:

```
management.endpoint.health.show-details=always
```

You get a structured breakdown of each subsystem.

This is extremely important for Kubernetes, AWS load balancers, and CI/CD pipelines because they rely on `/health` to decide whether an instance should receive traffic or be restarted.

◆ 6. The `/info` Endpoint — Showing Metadata About the App

`/info` allows you to expose versioning and build information about your application. This is helpful when multiple app versions are deployed in different environments.

Example configuration:

```
info.app.name=Order-Service
info.app.version=3.0
info.app.developedBy=Sheryians
```

Result:

```
{
  "app": {
    "name": "Order-Service",
    "version": "3.0",
    "developedBy": "Sheryians"
  }
}
```

This endpoint is mostly used for DevOps and debugging deployment issues.

◆ 7. The `/metrics` Endpoint — Monitoring Performance and Load

The `/metrics` endpoint provides deep insight into the performance of a running application.

Internally, Actuator uses **Micrometer**, a metrics library, to collect:

- JVM memory usage
- Garbage collection frequency
- Thread counts

- CPU load
- HTTP request count and response times
- Error rates

Example:

```
/actuator/metrics/http.server.requests
```

This shows:

- How many requests came
- How long each request took
- How many succeeded
- How many failed

Monitoring tools like **Grafana**, **Prometheus**, **Datadog**, **Splunk** rely heavily on this endpoint.

Metrics are the backbone of modern microservices observability.

8. The `/env` Endpoint — Understanding Active Configuration

The `/env` endpoint reveals everything about your application's runtime configuration:

- Environment variables
- System properties
- Application.properties values
- Config Server overrides
- Random values assigned by Spring Boot

This is extremely useful in situations where:

- The app picks the wrong DB URL

- The wrong port is active
- A property is overridden by another source

By checking `/env`, you can pinpoint exactly which property source supplied which value.

◆ 9. The `/beans` and `/mappings` Endpoints — Seeing the Internal Wiring

`/beans`

This displays every Spring bean that was created, including:

- bean name
- type
- dependency graph

It helps in solving:

- duplicate bean issues
- auto-configuration conflicts
- missing bean definitions

`/mappings`

Shows all the HTTP routes of your application and which method handles them.

Example:

```
GET /products → ProductController.getAll()
POST /orders → OrderController.create()
```

This is extremely helpful when debugging 404 errors or checking if controllers are mapped correctly.

◆ 10. The `/threaddump` and `/httptrace` Endpoints — Diagnosing Runtime Issues

◆ Thread Dump

`/threaddump` provides the current state of every thread inside the JVM.

It helps identify:

- Deadlocks
- Long-running tasks
- Bottlenecks
- Stuck background threads

◆ HTTP Trace

`/httptrace` shows the last 100 HTTP calls received by your application.

It allows you to inspect:

- Request method
- Response status
- Execution time
- Timestamps
- Headers

This is useful for detecting slow endpoints or repeated failing calls.

◆ 11. The `/shutdown` Endpoint — Controlled Application Shutdown

Spring Boot can shut down gracefully using:

```
POST /actuator/shutdown
```

But it is **disabled by default** due to high security risk.

To enable:

```
management.endpoint.shutdown.enabled=true  
management.endpoints.web.exposure.include=shutdown
```

This is mainly used by automated deployment scripts, not humans.

◆ 12. Securing Actuators in Production

If Spring Security is present, actuator endpoints become protected automatically.

Unauthorized request gives:

```
401 Unauthorized
```

Typical production setup:

- Run actuators on a separate port (ex: 9001)
- Restrict access to DevOps VPN or internal network
- Use a dedicated actuator user

This ensures sensitive operational data is not publicly exposed.

◆ 13. Adding Custom Health Indicators

Sometimes you want `/health` to include the status of:

- Payment gateway
- External APIs
- Message queues
- Internal microservices

You can create a custom health indicator by implementing `HealthIndicator`.

Example:

```

@Component
public class PaymentHealthIndicator implements HealthIndicator {
    public Health health() {
        boolean alive = checkPaymentGateway();
        return alive ?
            Health.up().withDetail("payment", "Available").build() :
            Health.down().withDetail("payment", "Not responding").build();
    }
}

```

This becomes part of `/health` automatically.

◆ 14. Before Actuator vs After Actuator

Area	Before Actuator	After Actuator
Health Check	Had to code manually	<code>/health</code> gives structured output
Metrics	Use external libs	Built-in <code>/metrics</code>
Bean Visibility	Print manually	<code>/beans</code>
Route Mapping	Debug manually	<code>/mappings</code>
Config Debugging	Print logs	<code>/env</code>
Thread Issues	JVM tools required	<code>/threaddump</code>
Graceful Shutdown	Manual scripts	<code>/shutdown</code>

Spring Boot Actuators convert hidden internal operations into clear, accessible insights.

🎯 Final Summary

Spring Boot Actuators are a collection of powerful, production-ready tools that allow developers and DevOps teams to observe and manage a running Spring Boot application.

They exist because once an application is deployed as a sealed JAR, we lose visibility into its internal state. Actuator endpoints restore this visibility in a structured, secure and standardized way. Through endpoints like `/health`, `/info`, `/metrics`, `/env`, `/beans`, `/mappings`, `/httptrace`, and `/threaddump`, we can monitor application health, performance, configuration, routing, and runtime behavior without modifying business logic or restarting the application.

Actuators are essential for microservices, cloud deployments, CI/CD, monitoring dashboards, and debugging production issues efficiently.

[@shivraj-gawande](#)